

JAN AADHAAR NL2SQL

A Local Retrieval-Augmented Natural Language to SQL Query Generation System

Technical project guide: vector embeddings, FAISS schema retrieval, RAG context construction, Ollama LLM generation, validation, optimization, and UI workflow.

Current Component	Implementation
SQL generation model	Ollama qwen2.5-coder:3b (laptop-friendly default)
Embedding model	Ollama nomic-embed-text
Vector search	FAISS IndexFlatIP, persisted locally
Demo database	SQLite through SQLAlchemy
Interface	Streamlit UI and Python CLI
Security posture	Single SELECT-only SQL validation gate

Prepared from the implemented project code | 25 May 2026

1. Project Purpose

This project lets a user ask database questions in ordinary language and receive a SQL query. It is designed around a Jan Aadhaar-style relational database containing family, citizen member, welfare, banking, and verification information.

A question such as **Show all female beneficiaries receiving pension in Jaipur district** is translated into a query that joins only the necessary tables and uses only the required columns.

The Core Design Constraint

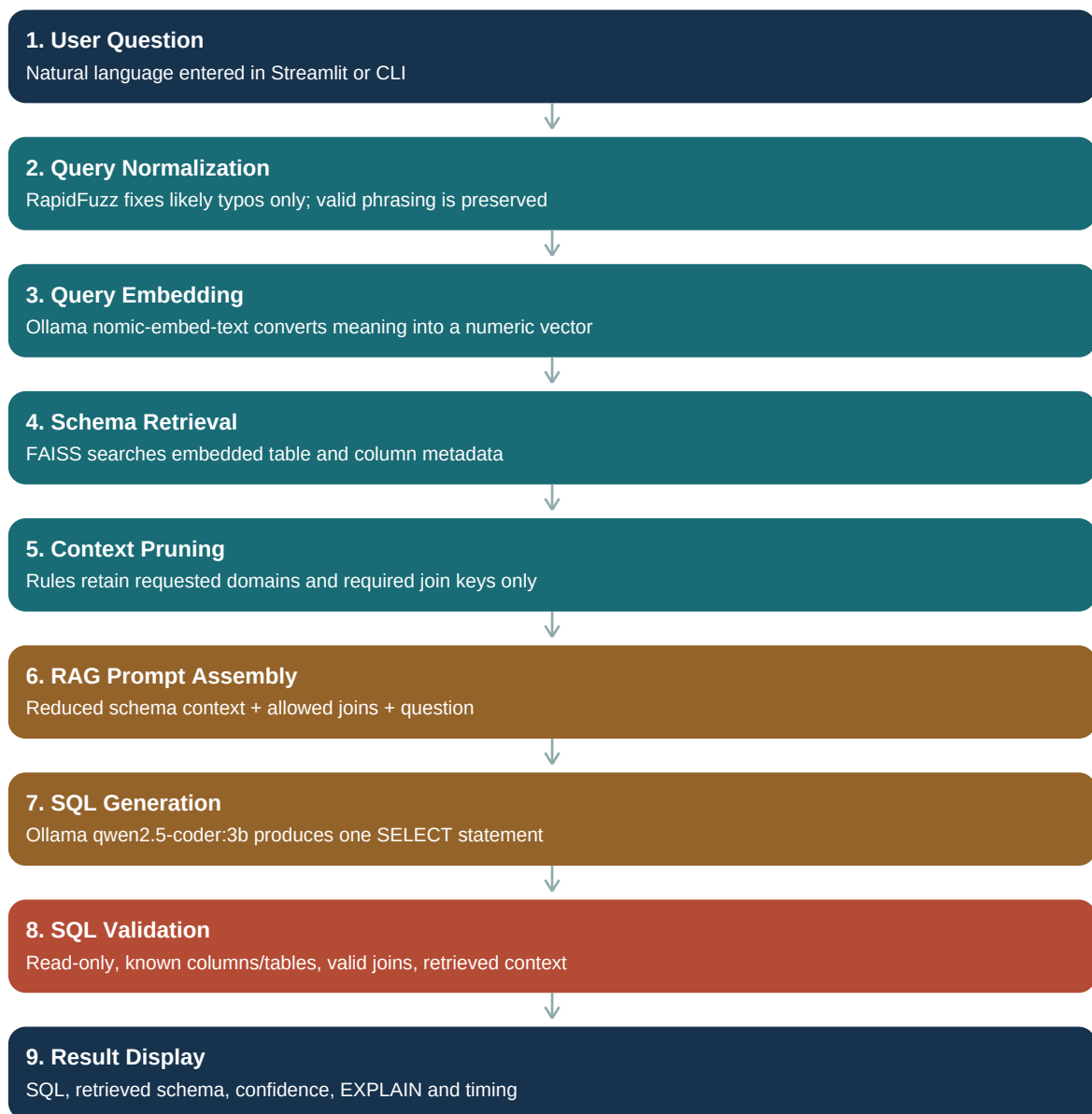
A real citizen registry can contain a very large schema and extremely large datasets. The full schema is therefore **not** sent to the language model for every request. Instead, the system searches a vector index of schema descriptions and sends a small, relevant schema context.

What the System Returns

- A generated, read-only SQL SELECT query.
- The tables and columns retrieved for the question.
- A schema retrieval confidence score.
- An optional SQLite EXPLAIN plan and planning/execution time.
- Visible spelling corrections when the input appears misspelled.

2. End-to-End Architecture Flowchart

The following flowchart separates the three AI-related activities: embedding meaning, retrieving relevant knowledge, and generating SQL.



3. What Vector Embeddings Are Doing

Simple Meaning

An embedding is a list of numbers representing the meaning of text. Texts with similar meaning produce vectors that are close to one another. For this project, embeddings do not represent citizen records; they represent **schema metadata** and the user's question.

Where Embeddings Are Used

Moment	Text Embedded	Why
Index build	Table and column documents	Prepare a searchable map of the database schema.
Each question	Normalized user question	Find columns semantically related to the user's intent.

Schema Documents

Each table and each column becomes a short semantic document in **embeddings/faiss_store.py**. A column document carries its physical name, business meaning, description, type, aliases, and useful sample values.

```
column member.gender
semantic name: gender
description: Citizen gender male female other;
              boy or boys means Male; girl or girls means Female
type: string
aliases: gender, sex, male, female, boy, boys, girl, girls
sample values: Male, Female, Other
```

This also supports legacy schemas. If the physical database really has a misspelled column such as **member.gendr**, its metadata can set **semantic_name='gender'**. Retrieval understands the clean meaning, while generated SQL still uses the real physical column.

Embedding Model and Similarity

- Model: **nomic-embed-text**, executed locally through Ollama.
- Vectors are normalized with NumPy after they are produced.
- FAISS uses **IndexFlatIP**; inner product on normalized vectors acts like cosine similarity.
- The persisted files are **data/schema.faiss** and **data/schema_metadata.json**.

Why This Is Efficient

The index contains schema entries, not 8 crore citizen records. Its size grows with the number of tables and columns, so searching it remains lightweight even when the underlying citizen database becomes very large.

4. Where RAG Is Used

Meaning of RAG in This Project

Retrieval-Augmented Generation (RAG) means the LLM is not expected to remember or guess the schema. Before SQL generation, the application retrieves relevant schema knowledge and adds it to the model prompt.

This is **schema RAG**, not record RAG: FAISS stores table/column descriptions and aliases, not personal citizen data.

Retrieval Pipeline

Stage	Implementation	Example for 'boys above 21 in Jaipur'
Vector recall	FAISS similarity search	Likely discovers age, gender, district.
Lexical anchors	Aliases and known Rajasthan districts	boys -> gender; Jaipur -> district.
Domain gates	Exclude domains unless requested	No pension, eKYC, bank, or caste columns.
Join enrichment	Include required foreign keys	member.family_id and family.family_id.
Display enrichment	Name for list queries	member.member_name.

Pruned Schema Context Example

For a demographic/location question, the retrieved context is deliberately compact:

Tables:
family
member

Columns:
family.district
family.family_id
member.age
member.family_id
member.gender
member.member_name

Relationship:
member.family_id = family.family_id

Columns such as **member.caste_category**, **verification.ekyc_status**, and **scheme_benefits.pension_eligibility** are withheld unless the question actually requests those subjects. This reduces prompt size and decreases hallucination opportunities.

Why the Architecture Is Hybrid

Vector search is strong at meaning and synonyms, but production behavior also needs deterministic boundaries. The implementation combines vector retrieval with explicit domain gating and pruning. That provides semantic flexibility while preventing irrelevant sensitive domains from being exposed to the SQL generator.

5. Natural Language Cleanup and Domain Vocabulary

Spelling Correction

The `normalization/query_normalizer.py` module uses RapidFuzz before retrieval. It fixes likely misspellings of controlled terms and Rajasthan district names, but preserves correctly written natural language rather than rephrasing it.

Input Fragment	Normalized Fragment	Reason
femail	female	Known typo correction
beneficiaries	beneficiaries	Known typo correction
jaipor	Jaipur	District typo correction
female beneficiaries receiving pension	unchanged	Valid text must not be rewritten

Location Handling

The metadata includes the 41 Rajasthan districts used by the project. Explicit district aliases help FAISS and lexical retrieval recognize locations such as Jaipur, Jodhpur, and Bikaner. A generic fallback also treats text shaped like **in <place>** or **from <place>** as a potential district filter.

Business Meaning Versus Physical Schema

The column metadata separates the true SQL identifier from its human meaning. This matters when a legacy production schema contains non-standard or misspelled physical column names.

```
ColumnMeta(  
    table="member",  
    column="gendr",          # physical database column  
    semantic_name="gender",  # clean business meaning  
    description="Citizen gender male female other",  
    aliases=["gender", "male", "female", "boy", "girl"],  
)
```

With this setup the embedding index can match the word **boys** to gender semantics, and the prompt can correctly tell the LLM that SQL must reference the physical name **member.gendr**.

6. Relational Data Model

The demonstration schema is represented in SQLAlchemy and can be created in SQLite. Its design mirrors the main data domains expected in a Jan Aadhaar-like registry.

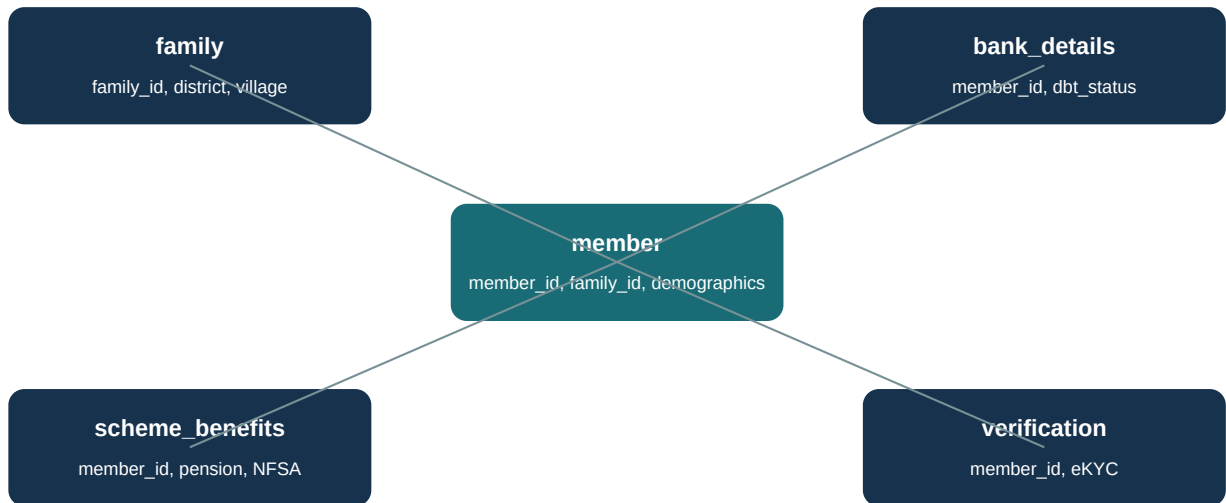


Table Responsibilities

Table	Purpose	Key Columns Used in Questions
family	Household and geographic location	family_id, district, block, village, jan_aadhaar_number
member	Citizen demographic and identity profile	member_id, family_id, member_name, age, gender, caste_category
scheme_benefits	Welfare eligibility and benefit states	member_id, nfsa_status, pension_eligibility, beneficiary_status
bank_details	DBT and banking information	member_id, dbt_status, ifsc_code
verification	eKYC and seeding states	member_id, ekyc_status, aadhaar_seeding_status

Declared Joins

```
member.family_id      = family.family_id
bank_details.member_id = member.member_id
scheme_benefits.member_id = member.member_id
verification.member_id = member.member_id
```

Indexes in the Demo Schema

- **ix_family_geo**: district, block, gram_panchayat, village.
- **ix_member_gender_caste_age**: gender, caste_category, age.
- **ix_scheme_pension_beneficiary**: pension_eligibility, beneficiary_status.
- **ix_verification_ekyc_aadhaar**: ekyc_status, aadhaar_seeding_status.

The demo SQLite database is for functionality demonstrations. At production citizen scale, a server-grade RDBMS or warehouse would replace SQLite while preserving the metadata/retrieval layer.

7. Prompt Building and LLM SQL Generation

Role of the LLM

The LLM is responsible for composing a SQL SELECT query from the user's intent and the reduced schema context. It does **not** choose the entire schema from memory and does **not** receive citizen records.

Setting	Current Value	Reason
Generation model	qwen2.5-coder:3b	Responsive local default on a laptop
Embedding model	nomic-embed-text	Local semantic representation of schema/query
Temperature	0	Reduce random variation in SQL
Context window	2048	Reduced RAG prompts do not require a large window
Output token cap	256	SQL should remain compact
Keep-alive	30 minutes	Reduce repeated model reload latency in Streamlit

Prompt Guardrails

- Generate exactly one read-only SELECT statement.
- Use only the supplied tables, columns, and relationships.
- Never invent a column or use SELECT *.
- Only use caste fields when caste is explicitly requested.

- Interpret boy/girl, age comparisons, and district language consistently.

These are model instructions. They improve generation, but they are not treated as the final security boundary. The validator is the enforced control.

8. Worked Example: From Question to SQL

Question

All boys above 21 in Jaipur

Step A: Normalization

The sentence is already valid, so it is left unchanged and no correction banner is shown.

Step B: Semantic Retrieval and Pruning

```
family.district
family.family_id
member.age
member.family_id
member.gender
member.member_name
```

Step C: Reduced RAG Prompt

The LLM receives only the two relevant tables, the six columns above, the valid relationship **member.family_id = family.family_id**, and the user's question.

Step D: Generated SQL

```
SELECT DISTINCT member.member_name
FROM member
JOIN family ON member.family_id = family.family_id
WHERE member.gender = 'Male'
      AND member.age > 21
      AND family.district = 'Jaipur';
```

Step E: Validation

Check	Outcome
Single SQL statement	Pass
Read-only SELECT only	Pass
Tables present in retrieved context	Pass: member, family
Columns present in retrieved context	Pass
Declared join relationship	Pass
No dataset modification command	Pass

Step F: Optional Planning

The UI may request an EXPLAIN QUERY PLAN report. If actual timing is selected, only the already validated SELECT is executed against the demo database to measure execution behavior.

9. SQL Validation and Dataset Safety

A text-to-SQL application must treat generated SQL as untrusted output. The validator in **validation/sql_validator.py** is positioned after every generation attempt and before optimization or optional execution.

Enforced Read-Only Policy

SQL Pattern	Allowed?	Reason
SELECT member.member_name FROM member;	Yes	Single read-only query
UPDATE member SET age = 18;	No	Write statement
DELETE FROM member;	No	Deletes records
DROP TABLE member;	No	DDL command
SELECT ...; DELETE FROM member;	No	Multiple statements and write command
SELECT ... INTO backup_member ...;	No	Creates/writes output table

Other Validation Checks

- Unknown table names are rejected.
- Unknown physical columns are rejected.
- Columns not retrieved by the RAG context are rejected.
- Joins outside declared relationships are rejected.
- Qualified column references require that their table be included in FROM or JOIN.
- SQL aliases are resolved before join and column validation.

Retry Loop

If a generated query fails validation, its error is inserted into the next prompt and Qwen is allowed to regenerate. The application permits at most three attempts. If no valid SQL is produced, the application returns no usable SQL rather than exposing unsafe output.

Safety Boundary Clarification

The project currently generates and optionally runs SELECT queries against a demo SQLite database. For real citizen information, production deployment should also add authorization, sensitive-column masking, query timeouts, row limits, auditing, and read-only database credentials.

10. Optimization and Local Performance

Current Latency Profile

FAISS schema search is small and local. The primary runtime cost is local LLM inference through Ollama. The system currently uses **qwen2.5-coder:3b** instead of the larger 7B model to favor laptop responsiveness.

Implemented Speed Measures

Measure	How It Helps
Reduced schema context	Fewer prompt tokens and less SQL confusion.
qwen2.5-coder:3b default	Smaller local generation workload than 7B.
num_ctx = 2048	Avoids reserving excessive context for short SQL prompts.
num_predict = 256	Prevents long unnecessary model output.
keep_alive = 30m	Keeps the model resident during a Streamlit session.
Persisted FAISS index	Schema embeddings are not rebuilt per normal query.

EXPLAIN and Index Recommendations

The optimization layer can call SQLite's **EXPLAIN QUERY PLAN** after SQL is validated. It can also flag referenced non-indexed columns as candidates for indexing if they are repeatedly used as filters.

Scaling to a Large Citizen Database

- Replace demo SQLite with PostgreSQL, SQL Server, Oracle, or a suitable analytical store.
- Run generated SQL with read-only credentials and strict query timeouts.
- Use partitioning/indexing strategies for district, block, benefit status, and update dates.
- Keep FAISS focused on schema metadata; do not embed sensitive citizen rows for this workflow.
- Introduce authorization-aware retrieval so only permitted columns can enter the LLM prompt.

11. Interfaces and Operations

Streamlit UI

The Streamlit interface provides a question text box, generated SQL display, retrieved tables and columns, retrieval confidence, execution plan, execution time, spelling corrections, and optional setup actions.

Sidebar Control	Purpose
Pull missing Ollama models	Allows required local models to be downloaded if absent.
Execute generated query for timing	Executes only validated SELECT SQL for timing.
Seed demo database	Creates and loads the small SQLite demonstration dataset.
Rebuild schema index	Re-embeds schema metadata after aliases/descriptions change.

CLI

```
python app.py "All boys above 21 in Jaipur"
python app.py --seed-demo-db --build-index "Show pension beneficiaries in Jaipur"
```

When to Rebuild the FAISS Index

- After adding or changing a table or column description.
- After adding aliases, district values, or semantic names.
- After mapping a legacy physical column name to a business meaning.
- Not for each ordinary user query.

12. Evaluation and Testing

The evaluation module contains benchmark questions and expected SQL. It records exact-match status, schema accuracy, retrieval accuracy, and latency. Unit tests cover prompt constraints, schema retrieval/pruning, typo normalization, model detection, and SQL validation safety.

Evaluation Metric	Meaning
Exact match	Whether generated SQL text matches expected normalized SQL exactly.
Schema accuracy	Whether generated SQL passes schema and safety validation.
Retrieval accuracy	Whether required expected columns were included in retrieved context.
Latency	Elapsed runtime for retrieval, generation, validation and planning.

13. Codebase Map

Path	Responsibility
app.py	End-to-end orchestration, retry loop, CLI entry point.
config/settings.py	Models, paths, retry count, retrieval size, Ollama keep-alive.
database/models.py	SQLAlchemy relational models and indexes.
database/schema_metadata.py	Semantic schema documents, aliases, districts, relationships.
normalization/query_normalizer.py	RapidFuzz user-input typo correction.
embeddings/ollama_embeddings.py	Local embedding calls and vector normalization.
embeddings/faiss_store.py	Schema document creation, FAISS build/load/search.
retrieval/schema_retriever.py	Hybrid vector/lexical retrieval and domain pruning.
prompting/prompt_builder.py	Reduced-context SQL generation prompt and guardrails.
llm/ollama_client.py	Ollama model checking/pulling and Qwen generation.
validation/sql_validator.py	Read-only policy and schema/join/context validation.
optimization/query_optimizer.py	Validated EXPLAIN, timing, index recommendations.
ui/streamlit_app.py	Browser interface for questions and results.
evaluation/benchmark.py	Benchmark execution and result metrics.

14. Glossary

Term	Meaning in This System
Embedding	Numeric meaning representation of a schema document or user question.
FAISS	Local vector similarity index used to search schema metadata.
RAG	Retrieving schema knowledge before asking the LLM to generate SQL.
LLM	Qwen model that composes SQL from question plus retrieved schema context.
Semantic metadata	Column/table description, aliases, sample values, and business meaning.
Pruning	Removal of unrelated retrieved columns before prompting the LLM.
Validation	Enforced check that SQL is read-only and uses permitted schema.
EXPLAIN	Database-generated description of how a validated query would be executed.

Summary

The project is a local, retrieval-first Text-to-SQL application. Its central idea is that a language model generates better and safer SQL when it receives a carefully retrieved slice of schema knowledge rather than the full database design. Embeddings and FAISS locate that knowledge, RAG passes it into the prompt, Qwen writes SQL, and deterministic validation decides whether the SQL is safe enough to display or execute.